



Módulo 1: Creamos el proyecto

¿Qué es Spring Boot?

Spring es el framework Java más utilizado en el mundo empresarial. Proporciona todo lo necesario para construir aplicaciones: gestión de dependencias, acceso a datos, seguridad, APIs web, etc.

Spring Boot es una capa encima de Spring que elimina la configuración manual. Su filosofía es *convention over configuration*: hace elecciones razonables por defecto para que puedas centrarte en el código de tu aplicación y no en configurar el framework.

Antes de Spring Boot configurar una aplicación Spring requería decenas de ficheros XML. Spring Boot lo reduce a prácticamente cero.

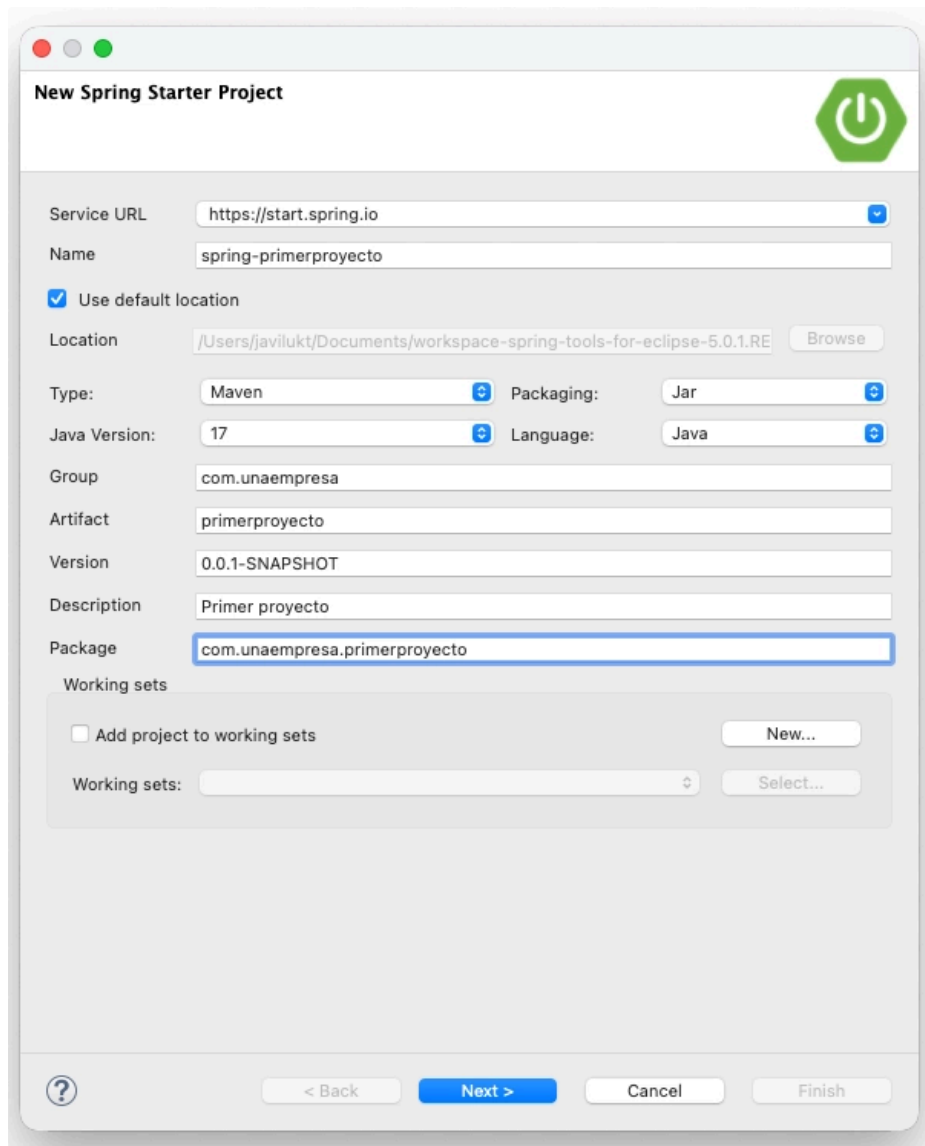
Crear el proyecto con Spring Starter Project

En Spring Tools Suite (STS) o Eclipse con el plugin de Spring, vamos a:

File → New → Spring Starter Project

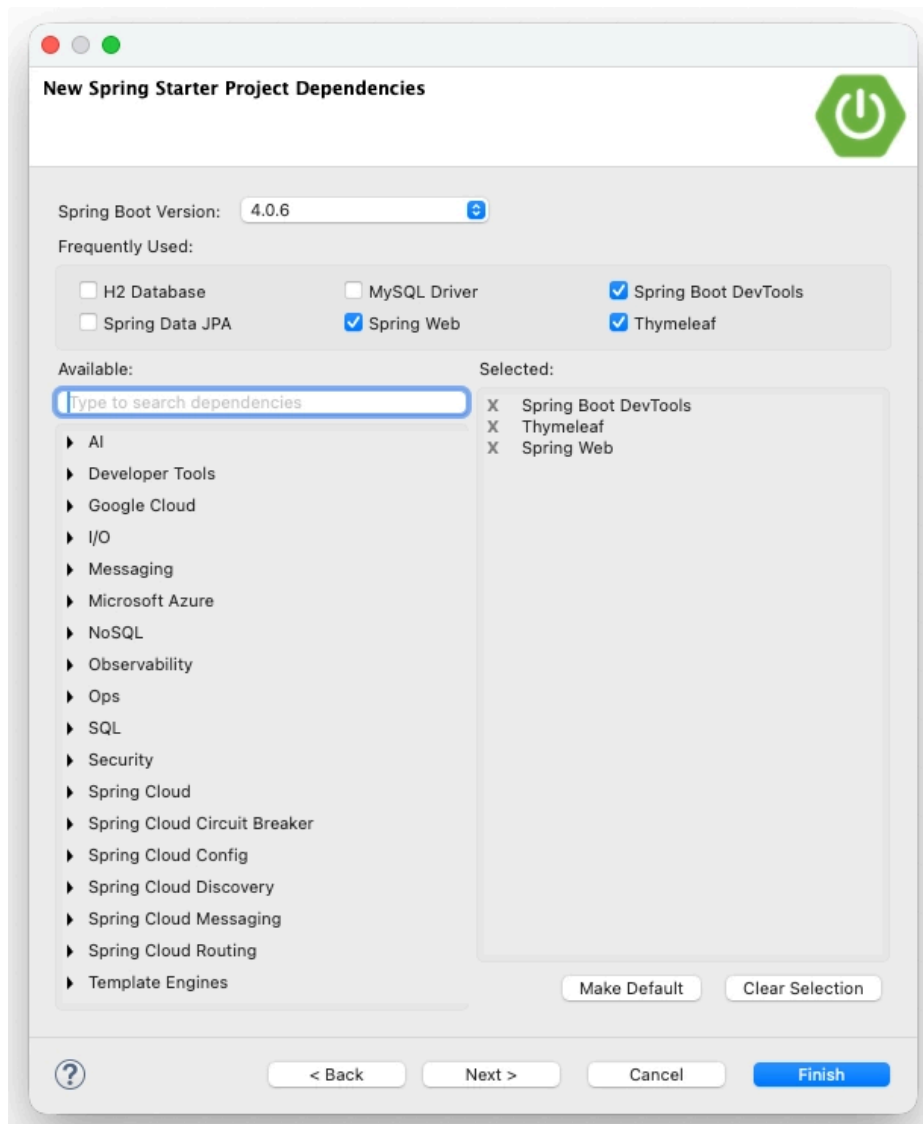
En el asistente rellenamos:

- **Name:** `spring-primerproyecto`
- **Group:** `com.unaempresa`
- **Artifact:** `primerproyecto`
- **Package:** `com.unaempresa.primerproyecto`
- **Java version:** 17



En la pantalla de dependencias seleccionamos:

- ☒ **Spring Web** — para crear controladores y mapear URLs
- ☒ **Thymeleaf** — motor de plantillas HTML
- ☒ **Spring Boot DevTools** — recarga automática en desarrollo



El `pom.xml` generado

El asistente nos genera este `pom.xml` :

En la línea de project, al principio, añadir una `s` a todos los `http`, para que ponga `https`. Antes o después dará problemas si no se pone.

```
<?xml version="1.0" encoding="UTF-8"?> <project
xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
<modelVersion>4.0.0</modelVersion> <parent>
<groupId>org.springframework.boot</groupId> <artifactId>spring-boot-
starter-parent</artifactId> <version>4.0.6</version> <relativePath/> <!--
- lookup parent from repository --> </parent>
<groupId>com.unaempresa</groupId>
<artifactId>primerproyecto</artifactId> <version>0.0.1-
SNAPSHOT</version> <name>spring-primerproyecto</name>
<description>Primer proyecto</description> <url/> <licenses> <license/>
</licenses> <developers> <developer/> </developers> <scm> <connection/>
<developerConnection/> <tag/> <url/> </scm> <properties>
<java.version>17</java.version> </properties> <dependencies>
<dependency> <groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-thymeleaf</artifactId> </dependency>
<dependency> <groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-webmvc</artifactId> </dependency>
<dependency> <groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-devtools</artifactId> <scope>runtime</scope>
<optional>true</optional> </dependency> <dependency>
<groupId>org.springframework.boot</groupId> <artifactId>spring-boot-
starter-thymeleaf-test</artifactId> <scope>test</scope> </dependency>
<dependency> <groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-webmvc-test</artifactId>
<scope>test</scope> </dependency> </dependencies> <build> <plugins>
<plugin> <groupId>org.springframework.boot</groupId> <artifactId>spring-
boot-maven-plugin</artifactId> </plugin> </plugins> </build> </project>
```

¿Por qué no ponemos versiones en las dependencias?

Porque heredamos de `spring-boot-starter-parent`. Este POM padre ya tiene definidas las versiones de todas las librerías del ecosistema Spring, garantizando que son compatibles entre sí. Solo necesitamos indicar el `groupId` y el `artifactId`.

Estructura de carpetas

Una vez creado el proyecto, la estructura es:

```
spring-primerproyecto/ ├── src/ | ├── main/ | | ├── java/ | | | └──
com/unaempresa/primerproyecto/ | | | └──
SpringPrimerproyectoApplication.java | | └── resources/ | | ├──
static/ ← CSS, JS, imágenes | | ├── templates/ ← plantillas
Thymeleaf (.html) | | └── application.properties | └── test/ | └──
java/ └── pom.xml
```

Donde:

- **src/main/java** — todo nuestro código Java
- **src/main/resources/templates** — las vistas HTML (Thymeleaf las busca aquí automáticamente)
- **src/main/resources/static** — recursos públicos: imágenes, CSS, JavaScript
- **application.properties** — configuración de la aplicación (puerto, base de datos, etc.)
- **pom.xml** — descriptor del proyecto Maven: dependencias y configuración de build

La clase principal

El asistente nos genera esta clase de entrada:

```
package com.unaempresa.primerproyecto; import
org.springframework.boot.SpringApplication; import
org.springframework.boot.autoconfigure.SpringBootApplication;
@SpringBootApplication public class SpringPrimerproyectoApplication
{ public static void main(String[] args) {
SpringApplication.run(SpringPrimerproyectoApplication.class, args);
} }
```

¿Qué hace **@SpringBootApplication** ?

Es una anotación compuesta que agrupa tres anotaciones:

- `@Configuration` — esta clase puede declarar beans de Spring
- `@EnableAutoConfiguration` — Spring Boot configura automáticamente todo lo que detecta en el classpath (por ejemplo, al ver Thymeleaf en el pom, configura el motor de plantillas solo)
- `@ComponentScan` — Spring escanea el paquete actual y todos sus subpaquetes buscando clases anotadas con `@Controller`, `@Service`, `@Repository`, etc.

Por eso todos nuestros controladores deben estar en el mismo paquete que la clase principal (llamado paquete base).

Si creamos un controlador fuera de `com.unaempresa.primerproyecto`, Spring no lo encontrará.

¿Y el `main` ?

`SpringApplication.run(...)` arranca el servidor embebido (Tomcat por defecto), carga el contexto de Spring y deja la aplicación escuchando peticiones HTTP. No necesitamos instalar ni configurar ningún servidor externo.

Arrancamos

Haz clic derecho sobre el proyecto → **Run As** → **Spring Boot App**.

En la consola verás algo como:

```

. ____ _ _ _ _ /\ / _ ' _ _ _ ( ) _ _ _ _ \ \ \ \ ( ( ) \ _ _ | ' _ |
'| | | ' \ _ ' | \ \ \ \ \ \ \ \ _ _ | | _ | | | | | | ( _ | | ) ) ) '
| _ _ | . _ | _ | | _ | | _ \ , | / / / /
=====|_|=====|_|_/ _ / _ / _ / [32m :: Spring Boot :: [39m [2m
(v4.0.6)[0;39m [2m2026-05-11T16:48:06.722+02:00[0;39m [32m INFO[0;39m
[35m53620[0;39m [2m--- [spring-primerproyecto] [ restartedMain]
[0;39m[36mc.u.p.SpringPrimerproyectoApplication [0;39m [2m:[0;39m
Starting SpringPrimerproyectoApplication using Java 25.0.2 with PID
53620 (/Users/javilukt/Documents/workspace-spring-tools-for-eclipse-
5.0.1.RELEASE/spring-primerproyecto/target/classes started by javilukt
in /Users/javilukt/Documents/workspace-spring-tools-for-eclipse-
5.0.1.RELEASE/spring-primerproyecto) [2m2026-05-
11T16:48:06.723+02:00[0;39m [32m INFO[0;39m [35m53620[0;39m [2m---
[spring-primerproyecto] [ restartedMain]
[0;39m[36mc.u.p.SpringPrimerproyectoApplication [0;39m [2m:[0;39m No
active profile set, falling back to 1 default profile: "default"
[2m2026-05-11T16:48:06.740+02:00[0;39m [32m INFO[0;39m [35m53620[0;39m
[2m--- [spring-primerproyecto] [ restartedMain]
[0;39m[36m.e.DevToolsPropertyDefaultsPostProcessor[0;39m [2m:[0;39m
Devtools property defaults active! Set 'spring.devtools.add-properties'
to 'false' to disable [2m2026-05-11T16:48:06.740+02:00[0;39m [32m
INFO[0;39m [35m53620[0;39m [2m--- [spring-primerproyecto] [
restartedMain] [0;39m[36m.e.DevToolsPropertyDefaultsPostProcessor[0;39m
[2m:[0;39m For additional web related logging consider setting the
'logging.level.web' property to 'DEBUG' [2m2026-05-
11T16:48:06.977+02:00[0;39m [32m INFO[0;39m [35m53620[0;39m [2m---
[spring-primerproyecto] [ restartedMain]
[0;39m[36mo.s.boot.tomcat.TomcatWebServer [0;39m [2m:[0;39m Tomcat
initialized with port 8080 (http) [2m2026-05-11T16:48:06.984+02:00[0;39m
[32m INFO[0;39m [35m53620[0;39m [2m--- [spring-primerproyecto] [
restartedMain] [0;39m[36mo.apache.catalina.core.StandardService [0;39m
[2m:[0;39m Starting service [Tomcat] [2m2026-05-
11T16:48:06.984+02:00[0;39m [32m INFO[0;39m [35m53620[0;39m [2m---
[spring-primerproyecto] [ restartedMain]
[0;39m[36mo.apache.catalina.core.StandardEngine [0;39m [2m:[0;39m
Starting Servlet engine: [Apache Tomcat/11.0.21] [2m2026-05-
11T16:48:06.993+02:00[0;39m [32m INFO[0;39m [35m53620[0;39m [2m---
[spring-primerproyecto] [ restartedMain]
[0;39m[36mb.w.c.s.WebApplicationContextInitializer[0;39m [2m:[0;39m Root
WebApplicationContext: initialization completed in 253 ms [2m2026-05-
11T16:48:07.079+02:00[0;39m [33m WARN[0;39m [35m53620[0;39m [2m---
[spring-primerproyecto] [ restartedMain]
[0;39m[36mion$DefaultTemplateResolverConfiguration[0;39m [2m:[0;39m
Cannot find template location: classpath:/templates/ (please add some
templates, check your Thymeleaf configuration, or set

```

```
spring.thymeleaf.check-template-location=false) [2m2026-05-11T16:48:07.104+02:00[0;39m [32m INFO[0;39m [35m53620[0;39m [2m---
[spring-primerproyecto] [ restartedMain]
[0;39m[36mo.s.boot.tomcat.TomcatWebServer [0;39m [2m:[0;39m Tomcat
started on port 8080 (http) with context path '/' [2m2026-05-11T16:48:07.106+02:00[0;39m [32m INFO[0;39m [35m53620[0;39m [2m---
[spring-primerproyecto] [ restartedMain]
[0;39m[36mc.u.p.SpringPrimerproyectoApplication [0;39m [2m:[0;39m
Started SpringPrimerproyectoApplication in 0.497 seconds (process
running for 0.768)
```

Abre el navegador en <http://localhost:8080> . Por ahora verás una página de error 404 — es normal, todavía no hemos creado ningún controlador.

```
Whitelabel Error Page This application has no explicit mapping for /error
May 11 16:49:03 CEST 2026 There was an unexpected error (type=Not Found,
org.springframework.web.servlet.resource.NoResourceFoundException: No st
org.springframework.web.servlet.resource.ResourceHttpRequestHandler.hand
at org.springframework.web.servlet.mvc.HttpRequestHandlerAdapter.handle(t
org.springframework.web.servlet.DispatcherServlet.doDispatch(DispatcherSe
org.springframework.web.servlet.DispatcherServlet.doService(DispatcherSe
org.springframework.web.servlet.FrameworkServlet.processRequest(Framework
org.springframework.web.servlet.FrameworkServlet.doGet(FrameworkServlet.
jakarta.servlet.http.HttpServlet.service(HttpServlet.java:622) at
org.springframework.web.servlet.FrameworkServlet.service(FrameworkServlet
jakarta.servlet.http.HttpServlet.service(HttpServlet.java:710) at
org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilt
org.apache.tomcat.websocket.server.WsFilter.doFilter(WsFilter.java:53) at
```